

Justificaciones de diseño iniciales.

1. Herencia:

- Donante: Humano, Juridico
- Bien: BienPercible, BienNoPercible, BienConEstado
- Necesidad: NecesidadExtraordinaria, NecesidadRecurrente

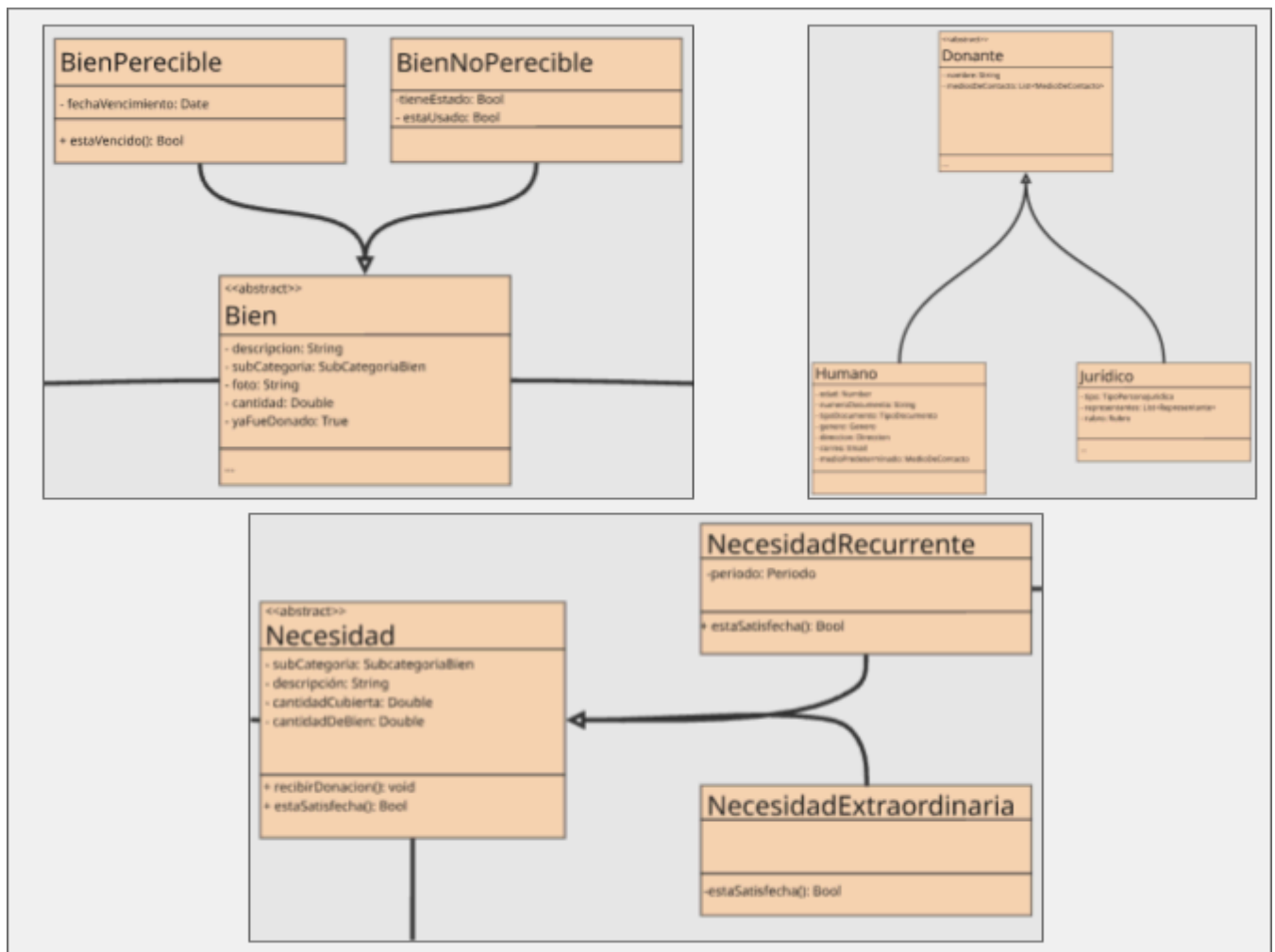
Decisión:

Se decidió usar herencia con clases abstractas y subclasses para modelar los distintos tipos de Donante, Bien y Necesidad.

Justificación:

En estos casos, cada tipo tiene características y comportamientos bastante distintos. Si se hubiera usado una sola clase para cada caso, habría muchos atributos que no aplican siempre y quedarían en null o habría que usar flags para diferenciarlos. Eso complica el modelo y lo hace menos claro.

Con herencia, cada subclase tiene solo lo que necesita y su propia lógica, lo que hace que el modelo sea más ordenado, más fácil de entender y más alineado con el dominio del problema.



2. Manejo de cantidades y unidades:

- UnidadDeMedida

Decisión:

Se decidió usar un Enum para las unidades de medida.

Justificación:

La idea fue poder manejar mejor las unidades de medida, ya que no todos los bienes se miden igual (por ejemplo, kilos, litros, unidades).

De esta forma estas quedan representadas bajo un conjunto cerrado y predefinido de valores, lo que logra un mayor control sobre la correcta medición de los bienes. Así, cada SubCategoriaBien se encargará de su propia de Unidad de Medida, evitando inconsistencias. Y cada bien se encargará de su propia cantidad.



3. Asignación:

- Donación

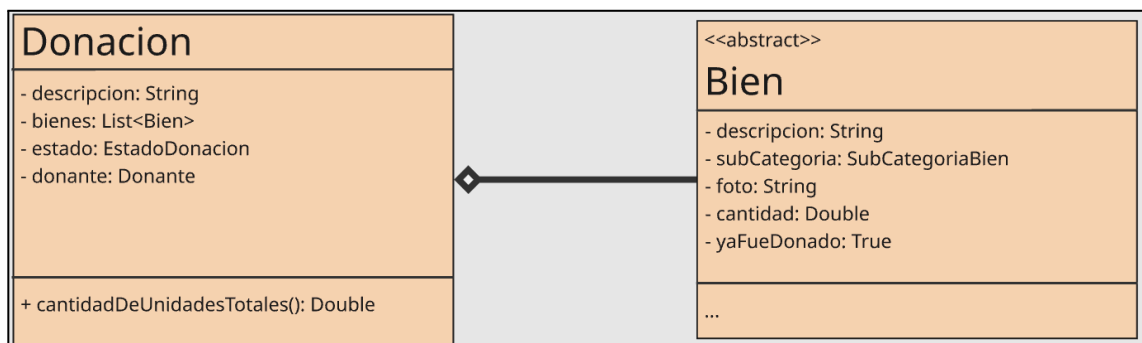
Decisión:

Se decidió que Donación sea una entidad propia, generada por el sistema y asociada a una sola subcategoría de bien.

Justificación:

Esto permite que cada donación sea lo más específica posible, lo cual es importante para el proceso de asignación (matching).

De esta forma, el sistema puede dividir los bienes según su subcategoría y asignarlos a las necesidades correspondientes. Además, cada donación puede tener su propio estado y evolucionar de manera independiente, lo que hace más flexible todo el proceso.



4. Datos que pueden repetirse:

- MedioDeContacto
- Representante

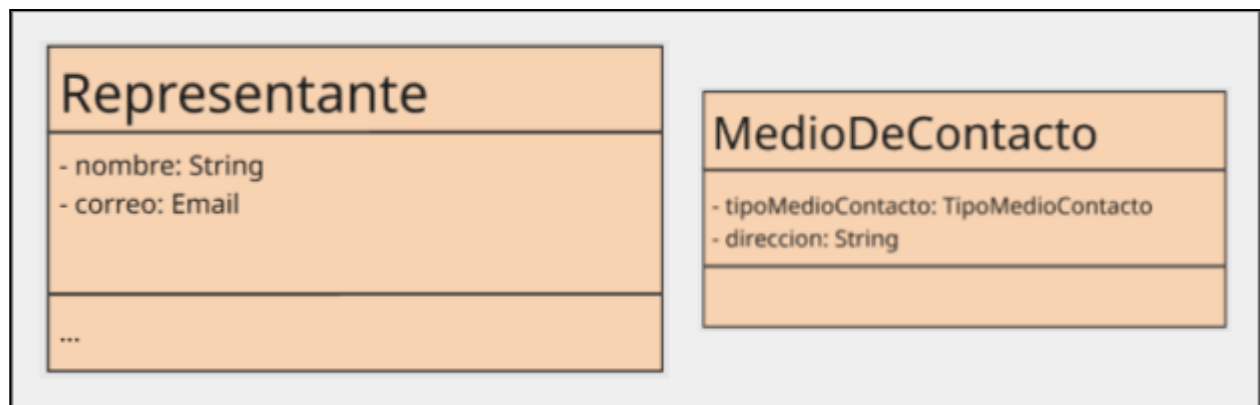
Decisión:

Se decidió modelar MedioDeContacto y Representante como clases independientes en lugar de poner esos datos como atributos dentro de otras clases.

Justificación:

Esto se hizo porque tanto los medios de contacto como los representantes pueden ser varios. Si se modelaban como atributos simples (por ejemplo, un solo teléfono o un solo representante), el diseño quedaba limitado.

Al usar clases separadas, se puede manejar cualquier cantidad de contactos o representantes sin problemas. Además, mejora la organización del modelo y permite agregar más información en el futuro si hace falta.



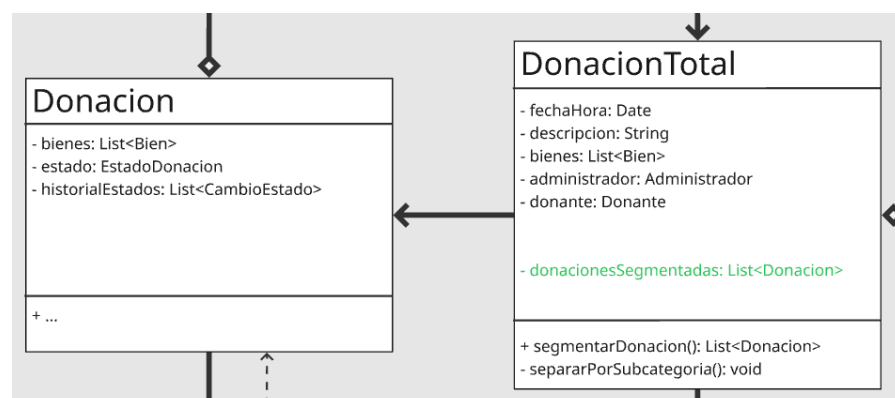
5. Donación / DonaciónTotal

Decisión:

Se decidió mantener una lista de bienes tanto en Donacion como en DonacionTotal.

Justificación:

Esto se hizo para conservar el detalle individual de cada bien que forma parte de una donación, incluso cuando varios bienes pertenecen a la misma subcategoría.



6. Eventos disparadores de notificaciones

Decisión:

Se decidió no crear una clase separada para los eventos que disparan las notificaciones.

Justificación:

Esto se hizo para reducir el acoplamiento entre el servicio de notificaciones y el resto de los servicios. Que el servicio de notificaciones maneje distintos “tipos” de eventos implicaría tener que cambiar el código del manejo de notificaciones, cuando lo único que se hace es enviar un mensaje notificación, independientemente del tipo de evento que sucedió. Los mismos servicios son los que construyen el mensaje antes de enviarlo al servicio de notificaciones para que lo despache.